

멀티프로세싱/멀티 프로그래밍

- 멀티프로세서 : 여러 프로세서(CPU 코어)가진 시스템, 코어들이 작업 나눠 수행 = 멀티프로세싱 ; 멀티프로그래밍 : 1개의 프로세서 -> 여러 프로그램이 시간을 나눠 사용
 - 배치작업(batch processing) : 한 PS가 종료 기다림 -> 다음 PS 수행
 - 시분할(Time-sharing) : 각 PS가 일정 시간 할당. 시간X? -> 다음 PS

프로세서(CPU), 프로세스

- 프로세서 : 실제로 프로그램이 실행되는 HW장치; MEM->CPU여만 실행 O
- 프로세스 : HDD->MEM으로 프로그램 올라와 실행O상태. CPU는 PS중 1개만 처리O
 - 프로세스 리스트 : OS -> 링크드 리스트로 PS 목록 작성;
 - 프로세스 구성 요소 :
 - 1) 프로그램 코드 영역 : 작성한 실제 프로그램 코드가 위치; 2) 데이터 영역 : 프로그램 수행에 필요한 데이터 3) 스택 영역 : 함수호출시 반환, 매개/지역 변수 저장; 함수호출시 스택이 쌓이고 종료시 스택에서 제거 4) 컨텍스트 : CPU가 PS 실행시 필요한 모든 정보(CPU 내 레지스터값 등); PS 중단 후 복원시 사용(시분할은 필수)

컨텍스트 스위치, 디스패처, 프로그램카운터(PC)

- 컨텍스트 스위치 : CPU가 실행중 PS의 상태(컨텍스트)를 PCB 저장 -> 다음 PS의 이전 상태를 PCB에서 로드
- 디스패처 : 여러 Ready 상태의 PS중 CPU에 할당할 PS를 선택해 Execute 상태로 전환
 - 원리 : (DP)PC값 변경 -> CPU가 실행할 PS 위치 지정. -> CPU는 DP가 PC값에 넣여주는 주소에서 가져와 실행. Ex) A:500, B:800, (DP->A?) -> PC=500, A 실행
- PC : CPU내 레지스터, 다음 실행할 PS의 메모리 주소 저장. 실행 PS만 execution, 나머지 Ready

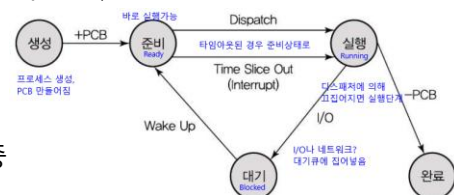
PCB(process control block)

- 정의 : OS가 PS를 제어/관리에 필요한 모든 정보를 담은 자료구조. PS 생성 때 마다 고유 PCB 생성. PS종료? -> PCB X. 같은 프로그램 여러번? -> 각기 다른 PCB
 - 내용 : PID, State, Priority, PC값, memory pointer(PS 사용중인 메모리 영역 정보), context(내부 레지값), 계정정보, 할당 자원, PPID/CPID, 포인터

포인터	프로세스 상태
프로세스 구분자	
프로그램 카운터	
프로세스 우선순위	
자용 레지스터 정보	
메모리 관리 정보	
할당된 자원 정보	
계정 정보	
PPID와 CPID	
...	

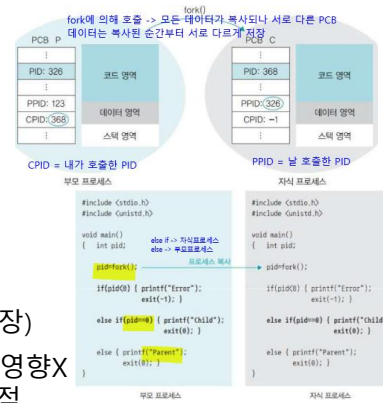
프로세스 상태

- Creation : HDD->MEM으로 로드, PS 생성
 - Ready : PS -> CPU 할당 대기. 언제든 실행O.
 - but, 다른 PS가 CPU 사용중. DP가 Ready의 PS중 하나 선택 -> CPU 할당
 - Execution(또는 Run) : 현재 PS 실행중. 싱글 코어? -> 1번에 1개의 PS만
 - Blocked(또는 waiting) : PS가 I/O 등 특정 이벤트를 기다림. 작업이 끝나면 HW에서 OS에 인터럽트 -> PS를 wake-up. 다시 Ready로; Termination : PS 실행 끝. PCB 삭제
 - Pause : MEM부족, HDD로 보냄; Suspended : MEM->HDD로 쫓겨남
 - Ready queue : 모든 Ready->Ready Queue; 단, 모든 Queue가 FIFO는X



Fork(), Exec()

- Fork : 기존 PS 복사, 동일 새 PS(자식 PS) 생성 system call. 호출PS?->부모 PS
→ PPS의 PCB 내용 복제. 단, 고유PID (CPID) 장점 : 추가작업X->자원 상속O
- Exec() : 같은 프로그램 코드 but, 새 코드로 자식 PS.
→ 자식PS가 부모PS와 다른 작업 수행? -> 새 프로그램으로 교체



멀티프로세싱과 멀티스레드

- 프로세스 : 가장 작은 실행 단위(OS입장). 독립적 메모리 공간 O
- 스레드(thread) : 한PS 내에서 동시에 여러 작업 수행하는 더 작은 단위(CPU입장)
→ 원리 : (멀티프로세싱) fork로 새PS -> 모든 자원 복사. 독립적 메모리, 타PS영향X
(멀티스레드) : 동일 PCB에서 각 스레드별. 단, stack, 레지는 스레드별 독립적
→ 멀티스레드 장점 : 코드/전역데이터 -> 메모리 낭비 ↓, 멀티코어에서 성능 ↑
→ 단점 : 오류전파 : 공유메모리 -> 한 스레드 오류? -> 모든 스레드 영향 전파. 종료

메모리 시스템

- 정의 : CPU가 직접 접근해 사용하는 주기억장치(물리적으로 DRAM)
- 메모리 주소 체계 : 0번지부터 시작, 1바이트(8비트)씩 증가. 0->8->16 ...번지
- 메모리 보호 : 운영체제+사용자 영역; 사용자가 OS 영역 침범X -> MMU로 HW적으로 경계선(ex : 0~100 번지는 OS)을 기준으로 접근 방지. OS에서 먼곳? -> 충돌 ↓
→ 절대주소 : 메모리의 물리적 주소, 상대주소(offset) : 기준에서 얼마큼?

메모리 할당 방식 : 메모리 어떻게 할당?

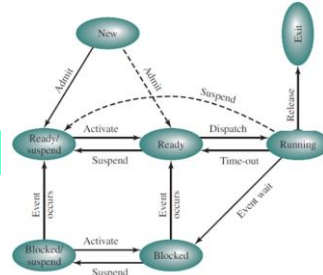
- 1) 고정분할 : 메모리 일정 크기(20kb 등)로 잘라 하나 할당 -> 연속X일수도 -> 불연속? 수행속도 ↓, 낭비 ↑
→ 장점 : 메모리 release후 Relocation, Reallocation 용이.
단점 : internal fragmentation: 요청크기 > 할당블록 크기, 낭비공간O
- 2) 가변분할 : PS MEM 요청->원하는 크기만큼 잘라 할당
→ 장점 : 내부파편화 X, 메모리 낭비 ↓, 연속적 -> 수행속도 ↑
단점 : external fragmentation : PS 종료, MEM 반환 -> MEM에 여러 작은 빈공간.
• 해결 : garbage collection : 흩어진 빈 공간 -> 1개의 큰 연속공간.
→ 메모리 할당 알고리즘 : first fit : 처음 본 빈공간, best fit : 가장 size비슷 공간 할당, worst fit : 가장 큰 size -> worst 쓰면 나중에 큰걸 되돌려받음. Best는 낭비 가능성.
• Buddy system : 고정+가변 분할 : 전체 MEM을 2^n 단위 분할. 3=2+1KB, 7=4+2+1KB
고정방식 : 나뉜 부분에서 JMP. MMU는 fixed 방식.

Virtual memory : HDD에 임시저장 후 나중에 MEM 사용 -> HDD 크기만큼 MEM 사용O

- Swap out/In : HDD에서 MEM에 들고고 내보냄; - CPU-cash 관계와 유사. 스왑=replacement
- Thrashing : 메모리가 너무 부족, 계속해서 스왑 인/아웃만 반복 -> 실제 작업보다 스왑만 반복. 성능 ↓

스케줄링

- 정의 : 레디 큐에서 읽고, 프로세스에 배치하기 위한 결정과정
→ 필요성 : **한정된 리소스, 여러 PS가 동시에 요구** -> 누구에게 어떻게 리소스 배분?
- 스케줄링 단계 : (고수준 스케줄링) : sys내 전체 작업 수 조절. 동시에 실행중인 PS의 총 개수가 정해짐 (PC 돌릴 때 몇개 프로그램 동시?)



Process state and scheduling

- 프로그램 중지의 2가지 경우 :
1) 스스로 멈춤; 오래 걸릴 수행->blocked로 외부 입력 대기
2) 강제로 멈춤. 당장 재가동OK, 무언가 기다리지 않음
Ready상태 (Block 상태에서 조건 만족시 -> Ready)

스케줄링 요건

- 처리능력(Throughput) 최대화 : 단위시간당 완료되는 PS 수 ↑↑
CPU 이용률(Utilization) 극대화 : CPU 정해진 시간 내 100% 가동
반환시간(Turnaround time) 최소화 : PS가 시스템에 들어와서 종료될 때 까지 (system in->system out 시간)
대기시간(waiting time) 최소화 : ready queue에서 기다리는 시간 최소화
응답시간(Response time) 최소화 : 대화식 시스템에서 첫 응답까지 시간 ↓

스케줄링 목적

- **공평성(fairness)** : 특정 PS가 과다점유X; - **효율성(Efficient)** : 자원이 유휴시간X 사용
안정성(stability) : 우선순위->system 자원 독점/파괴에서 보호 + PS가 뒤섞임 X
반응시간 보장 : system은 적절한 시간 내에 응답해야함
무한연기방지(starvation prevention) : 특정 PS의 작업이 무한히 연기 X

CPU 집중 프로세스, 입출력 집중 프로세스

- CPU 집중 PS(CPU bound process) : 연산 사용 ↑↑ ex) 날씨 예측
입출력 집중 PS(I/O bound process) : 데이터 입출력 ↑↑ ex) 네트워크, 그래픽
→ 입출력 PS 우선배정시(or 전면 프로세스) 사용자 체감상
반응 ↑.But, response time 좋다고 항상 OK는 X (커널/OS PS ↑, 유저 PS ↓)

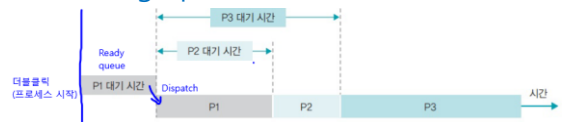


Preemptive/Non-preemptive Scheduling

- Preemptive scheduling : OS의 판단 -> PS 작업 중단, 새 작업OK
Non Preemptive : execution 상태로 CPU 사용 -> PS종료/block 전까진 방해X
→ 특수목적 SW는 방해X ex) 차 안전장치; 효율? -> preempt가 일어날 때 시간지연

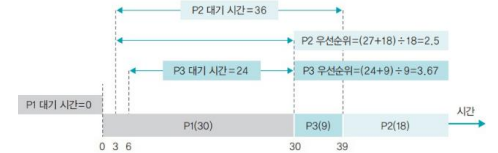
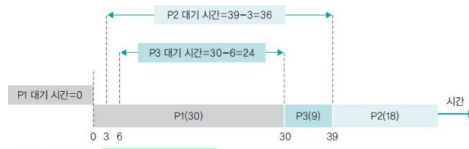
스케줄링 알고리즘 선택 기준 : Utilization vs Through put

- **평균 대기시간** : $\frac{\sum \text{대기시간}(P_n)}{n}$
turn around : 총 실행시간(실행+대기)
service time : $\sum \text{실행시간}$
**service time을 최적화X, waiting time만 OS가 조절->response time등을 조절
preemptive : 중간에 ready로 돌아감 -> waiting = $\sum \text{ready}$



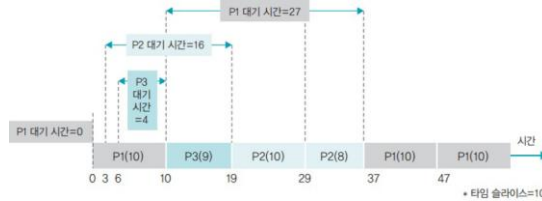
Non preemtive 스케줄링 기법

- **FCFS**(First come, first service) 스케줄링 : 레디큐 도착 순서대로 CPU 할당
→ 단점 : 처리시간 ↑ PS? → CPU 독점, 타 PS는 대기(=convoy effect), 특히 I/O 요청 시 쉬는 시간 ↑
- **SJF**(Shortest job first) : 레디큐 PS중 실행시간 짧은 작업부터 CPU 할당
→ 단점 : OS가 PS의 종료시간 예측 어렵. 작업시간 ↑? → 계속 대기(=Starvation)
해결책 : Aging : 오래된 순서에 맞춰 약간 보상 → 무한 대기X(PS의 양보 상한선)
- **HRN**(Highest Response Ratio Next) : $\frac{\text{대기시간} + \text{CPU사용시간}}{\text{CPU사용시간}} \rightarrow \text{대기시간} \propto \text{HRN}$



preemtive 스케줄링 기법

- **Round Robin** : PS가 타임 슬라이스 동안 작업 → 완료X 시 레디큐 끝에서 대기;
→ 장점 : FCFS의 콘보이 효과 완화; 단점 : TS 너무 ↓? → context switch에 시간낭비
- **SRT**(Short Remaining Time) : SJF의 Preemtive 버전, 로빈 방식에서 PS 선택시 남은 작업시간이 가장 적은 PS 선택

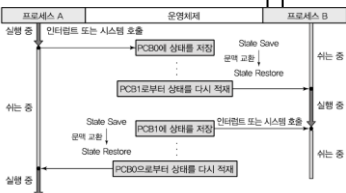
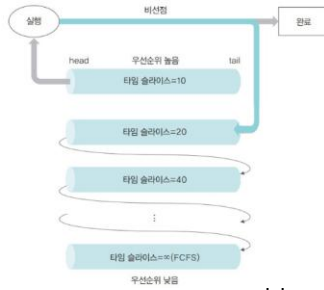


Priority 스케줄링, multi-level 큐

- **우선순위 스케줄링** : 각PS에 우선순위 → 가장 높 우선순위부터. 우선순위 동일? FCFS
→ (Static)PS 생성시 우선순위 지정, 변경X.; (Dynamic)PS 상태변화에 따라 변동
- **다단계 큐** : PS를 여러 ready 큐로 나눔 → 각 큐에 다른 우선순위 및 스케줄링 적용
→ Ex) OS PS 큐를 가장 ↑, 사용자 PS 큐를 ↓. 각 큐 안에서도 I/O → CPU Bound 순
→ 문제점 : Starvation : (높 우선순위 큐 → 새 PS?) → (낮 우선 큐 → CPU 할당X)
해결책 : (PS이동) 높우큐 → 할당시간X? → 낮우큐. or 낮우큐 오래? → 높우큐 (타임슬라이스 차등부여) : 각 큐마다 다른 TS. 높우큐 → 짧 TS, 낮우큐 → 긴 TS

Context switch

- 정의: CPU에서 현재 PS의 상태(context)를 저장. 다음 PS의 상태 불러와 로드하는 과정
→ PS 중단 → PCB 저장 → ... → PCB 저장된 레지값들을 CPU 레지에 로드 → 재작업 발생 시점 : Timeout, I/O 작업, PS종료시, 자발적 양보(PS 스스로 포기)
- 오버헤드 : CS 수행중 PCB값을 읽고 쓰는데 시간O. 잦은 CS는 utilization ↓ → HW적으로 여러 레지세트 → CS시 레지값을 MEM 대신 다른 레지에 저장



프로세스 동기화 및 통신의 종류

- 프로세스간 통신의 종류 (Inter Process Communication, IPC)

- PS 내부 데이터 통신 : 1개의 프로세서 내의 메모리. PS 내에 2개 이상 스레드, 스레드는 전역변수/파일로 통신
- PS간 데이터 통신 : OS -> 프로세서간 메모리 공유 X(용이X); 파일(여러 PS 공유O)/파이프(PS간 통신채널, 단방향(양방향은 2개 사용), 동일PPID 하) 사용
- 네트워크를 통한 데이터 통신(RPC) : 소켓사용. Target address가 다른 컴일 필요X

- 통신 방향에 따른 분류 :

양방향(full duplex, ex : 전화), 반양방향(half duplex, ex : 무전기), 단방향(unidex)

- 통신 구현 방식에 따른 분류 :

- 대기O 통신(blocking) : 데이터 받는 쪽은 도착까지 대기 (동기화O)
- 대기 X 통신(non-blocking) : 받는 쪽 : 데이터 도착했나? -> 직접 확인 (동기화X)

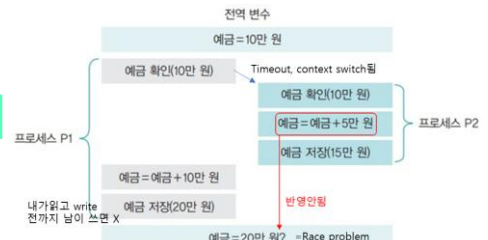


프로세스 동기화

- 어떤 지점에서 서로 시행 시점을 맞추는 것. Mutual exclusive : 공유자원 사용시 다른 PS가 사용X 하게 배제(1개의 PS만 공유자원 사용토록)
- 필요성 : Race condition 해소 : 2개 PS가 common resource 사용시 자원의 적절한 할당, critical section(프로그램 중 공유 자원을 접근하는 특정 부분, 1개의 PS만 OK)에 대한 mutual exclusive 보장

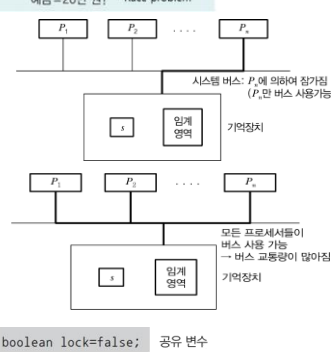
공유자원의 접근

- 공유자원 : 공공이용 -> 누가 언제 data 읽고 쓰는지에 따라 결과 상이 -> 경쟁조건
- 임계구역 : 구역내에선 PS들이 동시 작업X(1번에 1개만)



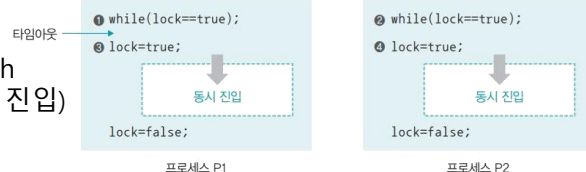
임계구역 구현방법

- 해결조건 :
 - 1) Mutual exclusive : (한 PS -> 임계)? -> 다른 PS -> 임계X
 - 2) bounded wating : PS -> 무한 대기 X(무한 임계X)
 - 3) progress flexibility : 한 PS -> 다른 PS의 진행방해X
- Bus Locking 방식 : HW적으로 내부 시스템 버스 통로를 1개로 제한, 한 Pn 사용시 나머지는 아무것도 X
- Spin-lock 방식 : lock 변수 검사 -> 허가된 경우 OK



전역변수로 임계구역 해결조건 고려한 코드

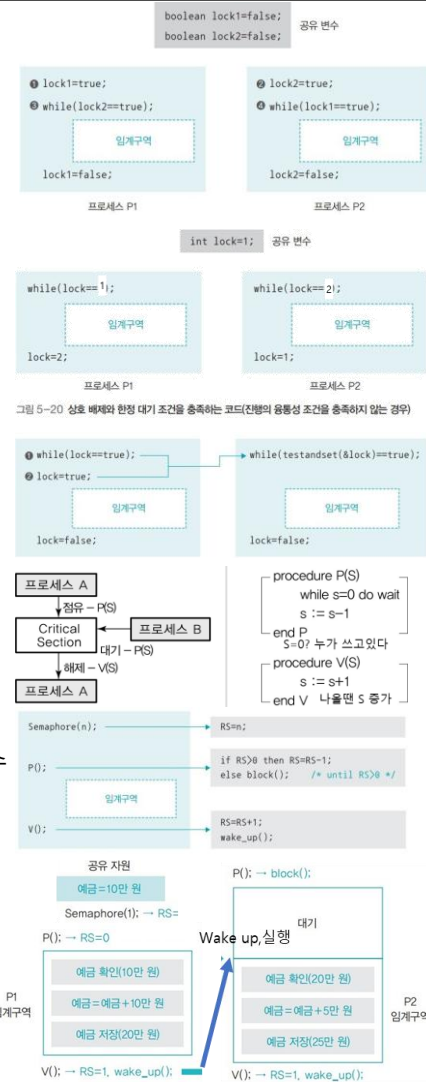
- Spin lock 방식; P1 : time out 지점에서 context switch -> P2 진행(3과 4에서 둘다 임계구역 진입) = Mutual exclusive 위배



Synchronization(계속)

임계구역 해결조건을 고려한 코드 (계속)

- Mutual exclusive 충족 코드 :
P1이 (1) 이후 타임아웃 -> P2로 context switch
-> P2가 (2) 이후 time out, context switch
->(3)을 실행하려 하지만 lock2 = true, 진행X,
(4)를 실행하려 하지만 lock1 = ture, 진행X
- Mutual exclusive, Bounded wait 다 충족 :
결정적 문제? Lock=1 -> 왼, Lock = 2 -> 오른;
if lock=2, P2가 실행되지 않는 한 P1은 1번만 실행
-> 한 PS가 다른PS 방해(progress flexibility 위배)
- 임계구역 문제의 HW적 해결 :
testandset() 으로 HW의 지원 -> while(lock=true)
와 lock=true를 동시 실행
(testandset() : 메모리를 읽고 1로 세트)



Semaphore

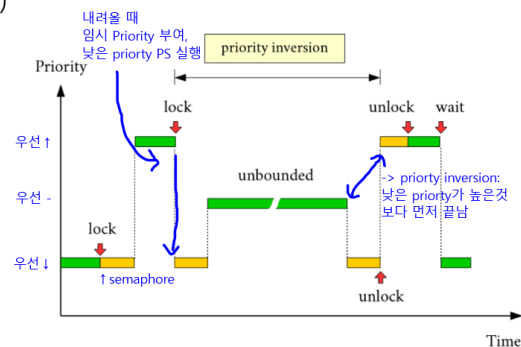
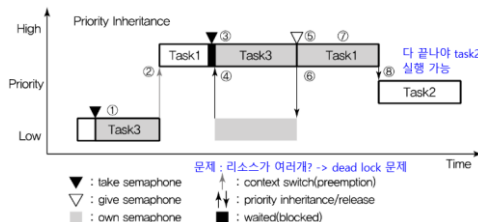
- Semaphore(n) -> n=k이면 k개의 PS가 CS 진입
P() : CS 진입 알림, V() : CS 나감 알림
-> OS가 하는 system call : CS에 들어가고 나가는 동안 OS는 context switch X
- Semaphore 내부 코드 :
Semaphore() : RS=n으로 초기화, RS는 가용 자원수
-> Semaphore -> P()? -> waitQueue로 진입
V()를 호출해야 ready로 진입
- 잘못된 용례 :
p()->p() : 세마포 대기 무한루프
v()->P() : ME 보장X, CS 보호X
-> 반드시 P()->V() 순으로 작성

Monitor 방식

- Semaphore X, os가 CS를 스스로 설정(공유 자원)을 OS에 걸쳐 사용 -> 더 안전; 단, 속도 ↓)

Priority Inversion

- Priority가 CS에 들어감 -> priority가 높은 PS가 대기; 낮은 PS가 빨리 끝내서 높은걸 쓰게 해주는게 중요
- 해결책 : priority inheritance



Dead lock

교착상태의 정의

- 2개 이상 PS가 다른 PS의 작업이 끝나기만 기다림, 작업진행X
- Starvation과 차이점 : OS가 잘못된 정책, 작업이 지연; dead lock은 진행 X

교착상태의 발생과 원인

- 발생 : 공유변수(spin lock 등), DB같은 응용 SW(네트워크 사용시;DB 일관성? -> 잠금)
- 원인(4가지를 모두 만족해야 dead lock 발생)
 - Mutual exclusion : Critical session 이유
 - Hold and wait : 자원을 점유중 -> 다른PS의 점유된 자원을 얻기 위해 대기중
 - Nonpreemption : 자원을 선점할수 없음
 - Circular wait : P1->P2-> ... Pn->P1의 순서로 점유한 자원 대기중

교착상태의 해결 (3조건 모두 OS가 가능해야함)

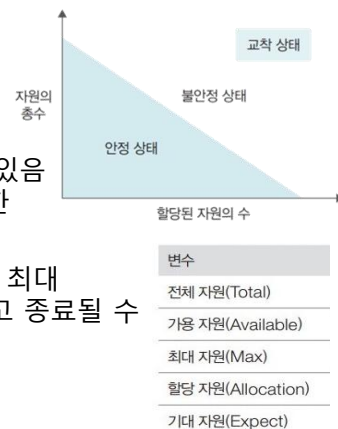
- Prevention : dead lock 4조건이 발생하지 않도록
- Avoidance : dead lock 발생할 가능성이 있지만 자원 신중 할당 -> sys 안전상태로
- Detection & recovery : dead lock 발생 허용. But, 교착상태가 있는지 검사(detection), 발생했다면 sys를 교착상태 이전으로 복구(recovery)

Dead lock Prevention (다음중 최소 1개 이상 부정)

- Mutual exclusive부정 : 모든 자원 공유 가능하게 -> 모두 만족할 충분한 자원? 불가능
- Hold and wait 부정: PS가 자원을 하나라도 점유시 자원 요청X. OS는 전부할당 아니면 주지 않음 -> 필요 리소스 모두 가져갈 수 있을 때 -> starvation : (리소스 많이 필요 -> 오랫동안 지연됨)
+ 내가 사용할 리소스 예측 -> 쉽지X
- Nonpreemption 부정 : 점유자원을 뺏어올 수 있도록(preemptive) 단 어떤 자원은 선점 자체가 불가능 ex) 프린터
- Circular wait 부정 : 모든 요청을 graph -> circular가 생김?, 있다면 회피 가능?
(PS 추가/제거 시 일일이 검사 -> 고비용)
+ 리소스마다 priority 정함, 자원 할당 시 낮은 것부터 차례로 -> 유연성 ↓

Dead lock Avoidance

- 교착상태 발생 가능성 O. but, 교착상태가 발생하지 않는 safe state에 머무르도록 자원 할당 동적 제어.
 - 현재 자원할당 상태와 각PS의 최대 자원요구량을 알고있음
-> 어느정도 자원이 남았을 때 안정상태를 OS가 최대한 확보하겠다
 - 은행원 알고리즘 : PS에 할당을 할 때, 현재 가용자원과 최대 요구량, 현재 할당량을 고려. 할당 이후에도 PS를 마치고 종료될 수 있는지 판단해 할당 여부 결정
 - $T = All - Ava$, $E = M - All$ 이 성립



Dead lock(계속)

Avoidance - 은행원 알고리즘(계속)

- 초기상태가 있을 때, 여기서 P2가 (1,0,1) 만큼 리소스 요구 -> (1,1,2)가 남아있으므로 배정 O. but, (0,1,1)배정했을 때 Dead lock 가능성 있나?
 - Dead lock avoidance면 고민. (0,1,1) P3 배정시 p4가 안끝남 -> hold & wait; 마찬가지로 P4도 끝나지 않는다.-> P2에 (0,1,1) 배정? 끝날수 있다 -> accept를 해도 dead lock 회피 O
 - P2는 accept Ok, P1,P3,P4의 경우?
 - > p1을 accept -> 현재 상황에선 4가지 조건 모두 O. but, P1~P4중 잘 따져서 dead lock 없이 할당 OK. 단, 원체 불능인 것도 O

	Claimed			Allocated			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	3	2	2	1	0	0	1	1	2
P2	6	1	3	5	1	1	P2 requests (1,0,1)		
P3	3	1	4	2	1	1			
P4	4	2	2	0	0	2			

	Claimed			Allocated			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	3	2	2	1	0	0	0	1	1
P2	6	1	3	6	1	2			

	Claimed			Allocated			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	3	2	2	1	0	0	6	2	3
P2	0	0	0	0	0	0			

Dead lock detection & recovery

- Detection : dead lock이 발생했나?(=PS가 일정 기간동안 멈췄나? Or 자원할당 그래프에 circular?)
- Recovery : 교착상태 -> sys를 dead lock에서 벗어나도록(dead lock 유발한 sys 강종)
 - PS 강종 방법 : 1) dead lock과 관련된 모든 PS 동시 종료 -> 비효율적. 2) dead lock과 관련된 PS중 순차적으로 종료

- 우선순위 낮 PS 종료 -> 작업시간 짧 PS 종료 -> 자원 많이 사용 PS 종료

→ PS 종료전략(avoidance) : wait-die wound-wait 전략

- Older PS : 수행시간 ↑, → : 리소스 요청 두 경우 모두 O가 먼저 끝나도록 배려
- Wait-die : O->Y : O가 wait, Y->O : Y가 스스로 die
- Wound-wait : O->Y : Y를 죽임, Y->O : Y가 wait

